

Summarization Service Test Documentation

Purpose

This document explains how to run and understand the test suite for the summarization microservice: a FastAPI service that either summarizes submitted text or refuses it when it violates a safety policy (harmful, illicit, or financial-advice content).

Local setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install flake8 pylint radon mutmut
```

Starting the service

```
python -m uvicorn app.main:app --reload
```

The service listens on `http://127.0.0.1:8000` by default.

Running tests

With the service running in one terminal, run the HTTP-level suite from another:

```
pytest tests/test_summarize.py -v
```

`tests/test_app_unit.py` and `tests/test_cli.py` import application code directly and don't need the service running:

```
pytest tests/test_app_unit.py tests/test_cli.py -v
```

Or run everything at once (the live-service tests will fail/error if uvicorn isn't running):

```
pytest -v
```

Test cases

Each entry below names the exact `project_inputs/` fixture file the test reads (where one is used) and the test function that reads it, so the fixture-to-test mapping is traceable without reading the source.

Health endpoint

- **Purpose:** confirm the service process is up and responding.
- **Fixture:** none — no input file needed for a liveness check.
- **Test:** `test_health_endpoint` in `tests/test_summarize.py`.
- **Expected result:** `GET /health` returns `200` with `{"status": "ok"}`.
- **Policy connection:** none; this is a liveness check.

Valid long-form summarization

- **Purpose:** confirm a normal, policy-clean document is summarized rather than refused.
- **Fixture:** `project_inputs/Long_text.txt`, read in full as the request body's `text` field.
- **Test:** `test_summarize_valid_long_text` in `tests/test_summarize.py`.
- **Expected result:** `POST /summarize` returns `200` with a non-empty `summary` string shorter than the input, and `refused: false`.

Response schema

- **Purpose:** lock the response contract so client code can rely on it.
- **Fixture:** none — uses an inline literal string, since the schema check doesn't depend on any specific fixture content.
- **Test:** `test_response_shape_schema_fields` in `tests/test_summarize.py`.
- **Expected result:** the response always contains `summary` (str), `word_count` (int, equal to `len(summary.split())`), `source_id` (str, echoing the request's `source_id`), `refused` (bool), `policy_code` (str or null), and `message` (str or null).

Harmful / illicit / financial-advice refusal

- **Purpose:** confirm requests seeking harmful, illegal, or guaranteed-return-style financial content are refused rather than summarized.
- **Fixture:** `project_inputs/forbidden_examples.json` — every example under all three top-level categories (`harmful_content`, `illicit_instructions`, `financial_advice`) is used; none are sampled or skipped.
- **Test:** `test_forbidden_content_is_refused` in `tests/test_summarize.py`, parametrized once per fixture example via `load_forbidden_cases()`.

- **Policy codes:** `safety_harm`, `safety_illicit`, `safety_financial` respectively (each example's expected code comes from the fixture file itself, not a hard-coded value in the test).
- **Expected result:** `refused: true`, `empty summary`, `word_count: 0`, `message: "refused: policy_violation"`, and the original unsafe text is not echoed back in the response.

Policies endpoint

- **Purpose:** confirm the list of supported policy codes is discoverable without reading the source.
- **Fixture:** none.
- **Test:** `test_policies_endpoint_returns_supported_policies` in `tests/test_summarize.py`.
- **Expected result:** `GET /policies` returns `200` with `{"policies": ["safety_harm", "safety_illicit", "safety_financial"]}`.

Unit test suite (`tests/test_app_unit.py`)

- **Purpose:** exercise `app.main`'s functions and endpoints in-process (via FastAPI's `TestClient`) so mutation testing can actually detect code changes — see "Mutation testing" below for why this file exists separately from `tests/test_summarize.py`.
- **Fixture:** none from `project_inputs/`. Keyword/category cases are inline literals in the test file itself (one case per entry in `HARMFUL_KEYWORDS` / `ILLICIT_KEYWORDS` / `FINANCIAL_KEYWORDS`), so each individual keyword — not just one example per category — is covered.

CLI test suite (`tests/test_cli.py`)

- **Purpose:** cover `scripts/run_tests.py`'s argument parsing and failure paths (missing dataset, unreachable service).
- **Fixture:** none from `project_inputs/` — tests write small throwaway JSON datasets to pytest's `tmp_path` fixture rather than reading the real `forbidden_examples.json`, since the goal is to test the CLI's own logic, not re-validate the dataset. The CLI itself defaults to `project_inputs/forbidden_examples.json` when run normally (see "Automation and CLI usage" below).

TDD workflow

`app/main.py`'s `/policies` endpoint was built test-first as a worked example of red/green/refactor:

1. **Red** — a failing test (`test_policies_endpoint_returns_supported_policies`) was committed against a `404` (`test: add failing policies endpoint test`).
2. **Green** — the smallest possible endpoint implementation was added to make it pass (`feat: implement policies endpoint`).

3. **Refactor** — the policy code literals were centralized into constants used by both the endpoint and the detection logic, with the full suite staying green throughout (`refactor: centralize supported policy definitions`).

The commit history for this sequence is saved at `artifacts/tdd-commit-history.txt` .

Complexity refactor

`project_inputs/review_comments.txt` asked for validation logic to be extracted out of the `summarize` handler and for repeated response construction to be centralized. `app/main.py` now has dedicated, docstringed helpers: `detect_policy_violation` , `resolve_source_id` , `build_refusal_response` , and `build_summary_response` , leaving the `/summarize` endpoint as a short coordinator.

Radon complexity reports before and after the refactor are saved under `artifacts/complexity/` (`radon-before.txt` / `radon-after.txt` / `comparison.txt`). The endpoint's core logic dropped from cyclomatic complexity B(7) to A(4)/A(2), and the file's average dropped from A(2.75) to A(2.0), with the full test suite green before and after.

Mutation testing

Mutation testing (via `mutmut`) mutates `app/main.py` (flips comparisons, changes constants, etc.) and checks whether the test suite notices. It needs tests that import the application in-process — the live-service tests in `tests/test_summarize.py` can't detect mutations because they talk to an already-running, unmutated uvicorn process. `tests/test_app_unit.py` exists for this: it imports `app.main` directly via FastAPI's `TestClient` .

```
mutmut run
mutmut results
```

Initial run: 39/75 mutants killed (score 0.52). After strengthening the unit tests (see `artifacts/mutation/summary.md` for the full breakdown — tautological constant comparisons, missing per-keyword coverage, and untested boundary conditions were the main gaps), the final run killed 70/75 (score 0.93). The remaining 5 survivors are documented as non-actionable: 2 are a `mutmut` /pytest exit-code compatibility gap (verified by reproducing the mutant manually) and 3 are genuinely equivalent mutants (unused `Optional` field defaults).

Test-data versioning

Every file under `project_inputs/` is hashed with SHA-256 and recorded in `project_inputs/manifest.json` , so it's possible to detect if a test input changed without anyone updating the corresponding test:

```
shasum -a 256 project_inputs/*
```

Model-run evaluation

`artifacts/model-evaluation-report.md` compares the candidate model runs in `project_inputs/candidate_model_runs.json` on safety pass rate and utility score, and records which run was selected and why (safety is weighted as the primary criterion for this service, since a false negative here means unsafe content gets summarized instead of refused).

Automation and CLI usage

`scripts/run_validation.sh` is the one-shot CI entry point: it starts uvicorn, waits for `/health`, runs `pytest/flake8/pylint/radon` against the running service, archives the results to a tarball, and always stops the service on exit (even on failure).

```
./scripts/run_validation.sh
```

`scripts/run_tests.py` is a standalone CLI for replaying a forbidden-examples-style dataset against any running instance of the service (useful for smoke-testing a deployed environment, not just localhost):

```
python scripts/run_tests.py \  
  --base-url http://127.0.0.1:8000 \  
  --dataset project_inputs/forbidden_examples.json \  
  --timeout 5 \  
  --artifact-dir artifacts/cli-run
```

It exits non-zero if the dataset path is missing/invalid, the service is unreachable, or any case fails, and writes a `results.json` with a per-case pass/fail breakdown. `tests/test_cli.py` covers its argument parsing and both failure paths.

CI execution

A CI job should, in order: install dependencies, run `./scripts/run_validation.sh` (which covers tests + lint + complexity against a live instance), then optionally run `mutmut run` and `python scripts/run_tests.py` against a deployed environment as a post-deploy smoke check. Non-zero exit from any step should fail the job.

Automation Proof: Terminal Output

Captured directly from running the test/lint/mutation-testing commands against this repository. Not screenshots of a GUI — raw stdout, unedited except for removing internal bookkeeping markers.

```
$ pytest -v
```

```
platform darwin -- Python 3.9.6, pytest-7.4.0, pluggy-1.6.0 -- /Users/davidortiz/summarization-  
microservice/.venv/bin/python  
cachedir: .pytest_cache  
rootdir: /Users/davidortiz/summarization-microservice  
configfile: pyproject.toml  
plugins: anyio-4.12.1, hydra-core-1.3.5  
collecting ... collected 52 items
```

```
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[kill] PASSED [ 1%]  
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[suicide] PASSED [ 3%]  
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[bomb] PASSED [ 5%]  
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[terrorist] PASSED [ 7%]  
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[shoot] PASSED [ 9%]  
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[harming others] PASSED [ 11%]  
tests/test_app_unit.py::test_detect_policy_violation_harm_keywords[violent actions] PASSED [ 13%]  
tests/test_app_unit.py::test_detect_policy_violation_illicit_keywords[make drugs] PASSED [ 15%]  
tests/test_app_unit.py::test_detect_policy_violation_illicit_keywords[cook meth] PASSED [ 17%]  
tests/test_app_unit.py::test_detect_policy_violation_illicit_keywords[fake passport] PASSED [ 19%]  
tests/test_app_unit.py::test_detect_policy_violation_illicit_keywords[hack bank] PASSED [ 21%]  
tests/test_app_unit.py::test_detect_policy_violation_illicit_keywords[illegal] PASSED [ 23%]  
tests/test_app_unit.py::test_detect_policy_violation_illicit_keywords[unlawful] PASSED [ 25%]  
tests/test_app_unit.py::test_detect_policy_violation_financial_keywords[guaranteed profit] PASSED [ 26%]  
tests/test_app_unit.py::test_detect_policy_violation_financial_keywords[insider trading] PASSED [ 28%]  
tests/test_app_unit.py::test_detect_policy_violation_financial_keywords[surefire stock] PASSED [ 30%]  
tests/test_app_unit.py::test_detect_policy_violation_financial_keywords[100% return] PASSED [ 32%]  
tests/test_app_unit.py::test_detect_policy_violation_financial_keywords[guaranteed investment] PASSED [ 34%]  
tests/test_app_unit.py::test_detect_policy_violation_financial_keywords[financial decisions without  
proper guidance] PASSED [ 36%]  
tests/test_app_unit.py::test_detect_policy_violation_clean_text PASSED [ 38%]  
tests/test_app_unit.py::test_resolve_source_id_uses_provided_value PASSED [ 40%]  
tests/test_app_unit.py::test_resolve_source_id_generates_when_missing PASSED [ 42%]  
tests/test_app_unit.py::test_simple_summarize_empty_text PASSED [ 44%]  
tests/test_app_unit.py::test_simple_summarize_short_sentence_returned_verbatim PASSED [ 46%]  
tests/test_app_unit.py::test_simple_summarize_exactly_thirty_words_returned_untouched PASSED [ 48%]  
tests/test_app_unit.py::test_simple_summarize_thirty_one_words_truncated_to_thirty PASSED [ 50%]  
tests/test_app_unit.py::test_simple_summarize_over_thirty_words_capped PASSED [ 51%]  
tests/test_app_unit.py::test_build_refusal_response_fields PASSED [ 53%]  
tests/test_app_unit.py::test_build_summary_response_word_count_matches_summary PASSED [ 55%]  
tests/test_app_unit.py::test_build_summary_response_empty_summary_has_zero_word_count PASSED [ 57%]  
tests/test_app_unit.py::test_endpoint_health PASSED [ 59%]  
tests/test_app_unit.py::test_endpoint_root PASSED [ 61%]  
tests/test_app_unit.py::test_endpoint_policies PASSED [ 63%]  
tests/test_app_unit.py::test_endpoint_summarize_refuses_unsafe_text PASSED [ 65%]  
tests/test_app_unit.py::test_endpoint_summarize_accepts_safe_text PASSED [ 67%]  
tests/test_cli.py::test_parse_args_defaults PASSED [ 69%]  
tests/test_cli.py::test_parse_args_custom_values PASSED [ 71%]  
tests/test_cli.py::test_parse_args_timeout_is_numeric PASSED [ 73%]  
tests/test_cli.py::test_load_dataset_missing_path_raises PASSED [ 75%]  
tests/test_cli.py::test_load_dataset_reads_valid_json PASSED [ 76%]  
tests/test_cli.py::test_main_returns_nonzero_for_missing_dataset PASSED [ 78%]  
tests/test_cli.py::test_main_creates_artifact_dir_even_when_service_unreachable PASSED [ 80%]  
tests/test_summarize.py::test_health_endpoint PASSED [ 82%]  
tests/test_summarize.py::test_summarize_valid_long_text PASSED [ 84%]  
tests/test_summarize.py::test_response_shape_schema_fields PASSED [ 86%]  
tests/test_summarize.py::test_forbidden_content_is_refused[harmful_content-0] PASSED [ 88%]  
tests/test_summarize.py::test_forbidden_content_is_refused[harmful_content-1] PASSED [ 90%]  
tests/test_summarize.py::test_forbidden_content_is_refused[illicit_instructions-0] PASSED [ 92%]  
tests/test_summarize.py::test_forbidden_content_is_refused[illicit_instructions-1] PASSED [ 94%]  
tests/test_summarize.py::test_forbidden_content_is_refused[financial_advice-0] PASSED [ 96%]
```

```
tests/test_summarize.py::test_forbidden_content_is_refused[financial_advice-1] PASSED [ 98%]
tests/test_summarize.py::test_policies_endpoint_returns_supported_policies PASSED [100%]
```

```
$ radon cc app --show-complexity --average
```

```
app/main.py
```

```
F 68:0 simple_summarize - B (6)
F 54:0 detect_policy_violation - A (4)
F 63:0 resolve_source_id - A (2)
F 95:0 build_summary_response - A (2)
F 127:0 summarize - A (2)
F 83:0 build_refusal_response - A (1)
F 109:0 health - A (1)
F 115:0 policies - A (1)
F 121:0 root - A (1)
C 10:0 SummarizeRequest - A (1)
C 15:0 SummarizeResponse - A (1)
```

```
11 blocks (classes, functions, methods) analyzed.
Average complexity: A (2.0)
```

```
$ python scripts/run_tests.py --base-url http://127.0.0.1:8000 \ --dataset
project_inputs/forbidden_examples.json --timeout 5 \ --artifact-dir artifacts/cli-run
```

```
6/6 cases passed. Results: /tmp/cli-proof-run/results.json
```

```
$ mutmut run && mutmut results
```

```
To apply a mutant on disk:
  mutmut apply <id>
```

```
To show a mutant:
  mutmut show <id>
```

```
Survived 😬 (5)
```

```
---- app/main.py (5) ----
```

```
1-5
```


test_summarize.py

```
1 import json
2 from pathlib import Path
3
4 import httpx
5 import pytest
6
7 BASE_URL = "http://127.0.0.1:8000"
8
9 PROJECT_ROOT = Path(__file__).resolve().parents[1]
10 LONG_TEXT_PATH = PROJECT_ROOT / "project_inputs" / "Long_text.txt"
11 FORBIDDEN_PATH = PROJECT_ROOT / "project_inputs" / "forbidden_examples.json"
12
13
14 def load_forbidden_cases():
15     content = json.loads(FORBIDDEN_PATH.read_text(encoding="utf-8"))
16     cases = []
17     for category, examples in content.items():
18         for index, example in enumerate(examples):
19             cases.append(
20                 pytest.param(
21                     example["input"],
22                     example["policy_code"],
23                     id=f"{category}-{index}",
24                 )
25             )
26     return cases
27
28
29 def test_health_endpoint():
30     r = httpx.get(f"{BASE_URL}/health")
31     assert r.status_code == 200
32     assert r.json()["status"] == "ok"
33
34
35 def test_summarize_valid_long_text():
36     long_text = LONG_TEXT_PATH.read_text(encoding="utf-8")
37     payload = {
38         "text": long_text,
39         "source_id": "valid-document-001",
40     }
41     r = httpx.post(f"{BASE_URL}/summarize", json=payload)
42     assert r.status_code == 200
43     data = r.json()
44
45     assert isinstance(data["summary"], str)
46     assert data["summary"].strip()
47     assert len(data["summary"]) < len(long_text)
48     assert data["refused"] is False
49
50
51 def test_response_shape_schema_fields():
52     payload = {"text": "This is a short test document.", "source_id": "doc-456"}
53     r = httpx.post(f"{BASE_URL}/summarize", json=payload)
54     assert r.status_code == 200
55     data = r.json()
56
57     assert "summary" in data
58     assert "word_count" in data
59     assert "source_id" in data
60     assert "refused" in data
```

```

61     assert "policy_code" in data
62     assert "message" in data
63
64     assert isinstance(data["summary"], str)
65     assert isinstance(data["word_count"], int)
66     assert isinstance(data["source_id"], str)
67     assert isinstance(data["refused"], bool)
68
69     assert data["source_id"] == payload["source_id"]
70     assert data["word_count"] >= 0
71     assert data["word_count"] == len(data["summary"].split())
72     assert data["refused"] is False
73     assert data["policy_code"] is None
74     assert data["message"] is None
75
76
77 @pytest.mark.parametrize("unsafe_text,expected_policy_code", load_forbidden_cases())
78 def test_forbidden_content_is_refused(unsafe_text, expected_policy_code):
79     payload = {"text": unsafe_text, "source_id": "unsafe-doc-001"}
80     r = httpx.post(f"{BASE_URL}/summarize", json=payload)
81     assert r.status_code == 200
82     data = r.json()
83
84     assert data["summary"] == ""
85     assert data["word_count"] == 0
86     assert data["source_id"] == payload["source_id"]
87     assert data["refused"] is True
88     assert data["policy_code"] == expected_policy_code
89     assert data["message"] == "refused: policy_violation"
90     assert unsafe_text not in data["summary"]
91
92
93 def test_policies_endpoint_returns_supported_policies():
94     response = httpx.get(f"{BASE_URL}/policies")
95     assert response.status_code == 200
96     data = response.json()
97     assert "policies" in data
98     assert isinstance(data["policies"], list)

```

test_app_unit.py

```
1 import pytest
2 from fastapi.testclient import TestClient
3
4 from app.main import (
5     app,
6     build_refusal_response,
7     build_summary_response,
8     detect_policy_violation,
9     resolve_source_id,
10    simple_summarize,
11 )
12
13 client = TestClient(app)
14
15 # Assertions below compare against literal policy codes/messages rather than
16 # importing the constants from app.main: importing them would make a test
17 # that mutates POLICY_HARM (say) compare the mutated value to itself and
18 # pass trivially, hiding the mutation instead of catching it.
19
20 HARM_CASES = ["kill", "suicide", "bomb", "terrorist", "shoot", "harming others", "violent actions"]
21 ILLICIT_CASES = ["make drugs", "cook meth", "fake passport", "hack bank", "illegal", "unlawful"]
22 FINANCIAL_CASES = [
23     "guaranteed profit",
24     "insider trading",
25     "surefire stock",
26     "100% return",
27     "guaranteed investment",
28     "financial decisions without proper guidance",
29 ]
30
31
32 @pytest.mark.parametrize("keyword", HARM_CASES)
33 def test_detect_policy_violation_harm_keywords(keyword):
34     assert detect_policy_violation(f"some text about {keyword} here") == "safety_harm"
35
36
37 @pytest.mark.parametrize("keyword", ILLICIT_CASES)
38 def test_detect_policy_violation_illicit_keywords(keyword):
39     assert detect_policy_violation(f"some text about {keyword} here") == "safety_illicit"
40
41
42 @pytest.mark.parametrize("keyword", FINANCIAL_CASES)
43 def test_detect_policy_violation_financial_keywords(keyword):
44     assert detect_policy_violation(f"some text about {keyword} here") == "safety_financial"
45
46
47 def test_detect_policy_violation_clean_text():
48     assert detect_policy_violation("a perfectly normal sentence") is None
49
50
51 def test_resolve_source_id_uses_provided_value():
52     assert resolve_source_id("doc-1") == "doc-1"
53
54
55 def test_resolve_source_id_generates_when_missing():
56     generated = resolve_source_id(None)
57     assert isinstance(generated, str)
58     assert generated != ""
59
60
```

```

61 def test_simple_summarize_empty_text():
62     assert simple_summarize("") == ""
63     assert simple_summarize(" ") == ""
64
65
66 def test_simple_summarize_short_sentence_returned_verbatim():
67     assert simple_summarize("Hello world.") == "Hello world"
68
69
70 def test_simple_summarize_exactly_thirty_words_returned_untouched():
71     # Deliberate double space between the first two words: the <=30 branch
72     # must return the sentence as-is, while the truncating branch would
73     # rejoin words with single spaces and lose the double space.
74     words = ["word{i}" for i in range(30)]
75     sentence = "word0 " + " ".join(words[1:]) + "."
76     summary = simple_summarize(sentence)
77     assert summary == "word0 " + " ".join(words[1:])
78     assert len(summary.split()) == 30
79
80
81 def test_simple_summarize_thirty_one_words_truncated_to_thirty():
82     words = ["word{i}" for i in range(31)]
83     sentence = " ".join(words) + "."
84     summary = simple_summarize(sentence)
85     assert summary == " ".join(words[:30])
86     assert len(summary.split()) == 30
87
88
89 def test_simple_summarize_over_thirty_words_capped():
90     long_sentence = " ".join(f"word{i}" for i in range(40)) + "."
91     summary = simple_summarize(long_sentence)
92     assert summary == " ".join(f"word{i}" for i in range(30))
93     assert len(summary.split()) == 30
94
95
96 def test_build_refusal_response_fields():
97     response = build_refusal_response("doc-9", "safety_harm")
98     assert response.summary == ""
99     assert response.word_count == 0
100    assert response.source_id == "doc-9"
101    assert response.refused is True
102    assert response.policy_code == "safety_harm"
103    assert response.message == "refused: policy_violation"
104
105
106 def test_build_summary_response_word_count_matches_summary():
107     response = build_summary_response("doc-10", "three word summary")
108     assert response.summary == "three word summary"
109     assert response.word_count == 3
110     assert response.refused is False
111     assert response.policy_code is None
112     assert response.message is None
113
114
115 def test_build_summary_response_empty_summary_has_zero_word_count():
116     response = build_summary_response("doc-13", "")
117     assert response.summary == ""
118     assert response.word_count == 0
119
120
121 def test_endpoint_health():
122     r = client.get("/health")
123     assert r.status_code == 200
124     assert r.json() == {"status": "ok"}

```

```
125
126
127 def test_endpoint_root():
128     r = client.get("/")
129     assert r.status_code == 200
130     assert r.json() == {"message": "Summarization microservice is running"}
131
132
133 def test_endpoint_policies():
134     r = client.get("/policies")
135     assert r.status_code == 200
136     assert r.json() == {
137         "policies": ["safety_harm", "safety_illicit", "safety_financial"]
138     }
139
140
141 def test_endpoint_summarize_refuses_unsafe_text():
142     r = client.post(
143         "/summarize",
144         json={"text": "instructions for harming others", "source_id": "doc-11"},
145     )
146     data = r.json()
147     assert data["refused"] is True
148     assert data["policy_code"] == "safety_harm"
149     assert data["message"] == "refused: policy_violation"
150     assert data["summary"] == ""
151     assert data["word_count"] == 0
152     assert data["source_id"] == "doc-11"
153
154
155 def test_endpoint_summarize_accepts_safe_text():
156     r = client.post(
157         "/summarize",
158         json={"text": "A short safe sentence.", "source_id": "doc-12"},
159     )
160     data = r.json()
161     assert data["refused"] is False
162     assert data["policy_code"] is None
163     assert data["message"] is None
164     assert data["summary"] == "A short safe sentence"
165     assert data["word_count"] == 4
```

test_cli.py

```
1 import json
2 import sys
3 from pathlib import Path
4
5 import pytest
6
7 sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "scripts"))
8
9 import run_tests as cli # noqa: E402
10
11
12 def test_parse_args_defaults():
13     args = cli.parse_args([])
14     assert args.base_url == "http://127.0.0.1:8000"
15     assert args.dataset == Path("project_inputs/forbidden_examples.json")
16     assert args.timeout == 5.0
17     assert args.artifact_dir == Path("artifacts/cli-run")
18
19
20 def test_parse_args_custom_values():
21     args = cli.parse_args(
22         [
23             "--base-url",
24             "http://example.test:9000",
25             "--dataset",
26             "some/dataset.json",
27             "--timeout",
28             "12.5",
29             "--artifact-dir",
30             "some/artifacts",
31         ]
32     )
33     assert args.base_url == "http://example.test:9000"
34     assert args.dataset == Path("some/dataset.json")
35     assert args.timeout == 12.5
36     assert args.artifact_dir == Path("some/artifacts")
37
38
39 def test_parse_args_timeout_is_numeric():
40     args = cli.parse_args(["--timeout", "3"])
41     assert isinstance(args.timeout, float)
42     assert args.timeout == 3.0
43
44
45 def test_load_dataset_missing_path_raises(tmp_path):
46     missing = tmp_path / "does_not_exist.json"
47     with pytest.raises(FileNotFoundError):
48         cli.load_dataset(missing)
49
50
51 def test_load_dataset_reads_valid_json(tmp_path):
52     dataset_path = tmp_path / "dataset.json"
53     dataset_path.write_text(json.dumps({"category": []}), encoding="utf-8")
54     assert cli.load_dataset(dataset_path) == {"category": []}
55
56
57 def test_main_returns_nonzero_for_missing_dataset(tmp_path):
58     exit_code = cli.main(
59         [
60             "--dataset",
```

```

61         str(tmp_path / "missing.json"),
62         "--artifact-dir",
63         str(tmp_path / "artifacts"),
64     ]
65 )
66 assert exit_code == 1
67
68
69 def test_main_creates_artifact_dir_even_when_service_unreachable(tmp_path):
70     dataset_path = tmp_path / "dataset.json"
71     dataset_path.write_text(
72         json.dumps({"category": [{"input": "hello", "policy_code": "safety_harm"}]}),
73         encoding="utf-8",
74     )
75     artifact_dir = tmp_path / "artifacts"
76
77     exit_code = cli.main(
78         [
79             "--base-url",
80             "http://127.0.0.1:1",
81             "--dataset",
82             str(dataset_path),
83             "--timeout",
84             "1",
85             "--artifact-dir",
86             str(artifact_dir),
87         ]
88     )
89     assert exit_code == 1
90     assert artifact_dir.exists()

```